

Architecture Justification

Pharmacy Expiry & Re-order Dispatch Engine (Problem Statement #15)

Architecture Selection

We chose Layered Architecture for the Pharmacy Expiry & Re-order Dispatch Engine.

The system is organised into a Presentation layer (Clerk UI, Supplier Portal), a Business layer (Inventory Manager, FEFO Selector, Stock Monitor, Purchase Order Service) and a Data layer (Database). Each layer only uses the services of the layer below it, through defined interfaces.

Reason 1: Clear separation of pharmacy rules

The FEFO rule is the core business logic of the system and is used by both dispensing (UC-02) and the picking list (UC-04). Placing it in a single FEFO Selector component in the business layer means the rule is written and changed in one place, independent of the screens and the database. The same applies to threshold checking (Stock Monitor) and purchase-order handling.

Reason 2: Strong data consistency at a modest scale

Every dispense must deduct stock and write an audit record together, and stock levels drive automatic re-ordering, so the data must always be consistent. A single shared data layer keeps this simple. Microservices would split inventory and orders into separate databases, adding data-consistency and operational complexity that a single hospital pharmacy does not need, while their main benefit (independent scaling) is unnecessary at this load.

Security Advantage

The presentation layer can never access the database directly; every request must pass through the business layer, where role-based checks are applied. A supplier using the Supplier Portal can only reach the IPOStatus interface and cannot touch inventory or dispensing. All writes go through IDataAccess, which appends to an immutable audit log (NFR-002), and purchase orders leave the system only over HTTPS/TLS (ISupplierDispatch).

Performance Benefit

Business components run in the same application, so calls such as Stock Monitor to Purchase Order Service (IPurchaseOrder) are in-process method calls with no network hop between services. This keeps the path from detecting a threshold breach to dispatching a purchase order short, supporting the 2-second dispatch target in NFR-001.

Alternatives Considered

Microservices: rejected due to operational complexity and data-consistency challenges. Client-Server: simple, but would mix business logic into the server without clear separation, making the FEFO and re-order rules harder to maintain and test.